

数据结构查找学习报告

成员分工

王枫：代码和学习报告编写

沈子力：演讲，书本文字提取

秦朗：

一、基于树的查找

线性表查找把记录放在数组或链表中；树表查找则把记录组织成树形结构，并根据关键字大小沿某一条分支继续查找。这样既能保持一定的查找效率，又比有序顺序表更方便插入和删除。

1. 二叉排序树

二叉排序树又称二叉查找树。它或者是一棵空树，或者满足以下性质：

1. 若左子树非空，则左子树上所有结点的关键字均小于根结点关键字。
2. 若右子树非空，则右子树上所有结点的关键字均大于根结点关键字。
3. 左、右子树也分别是二叉排序树。
4. 重要性质：**中序遍历二叉排序树可以得到递增有序序列**。若采用逆中序遍历，也就是先遍历右子树，再访问根结点，最后遍历左子树，则可以得到递减有序序列。

这是一个递归定义。由于左子树小于根、右子树大于根，因此二叉排序树的查找过程很像折半查找：每到一个结点，就根据比较结果只选择一侧子树继续查找。

二叉排序树的结点结构可以写成：

```
typedef struct node {
    KeyType key;
    struct node *lchild, *rchild;
} BSTNode, *BSTree;
```

二叉排序树的查找

二叉排序树的递归查找代码如下：

```
BSTree SearchBST(BSTree bst, KeyType key) {
    if (!bst || bst->key == key)
        return bst; // 可合并
    else if (key < bst->key)
        return SearchBST(bst->lchild, key);
    else
        return SearchBST(bst->rchild, key);
}
```

递归写法很贴近定义：空树表示失败；等于根结点则成功；小于根结点去左子树；大于根结点去右子树。

非递归用循环本质相同。

二叉排序树的插入与创建

二叉排序树的插入总是把新结点插到叶子位置。若树为空，新结点成为根；若树非空，则从根开始比较，小于当前结点就进入左子树，大于当前结点就进入右子树，直到找到空位置。

教材中的递归插入代码可以整理为：

```
void InsertBST(BSTree *bst, KeyType key) {
    BSTree s;

    if (*bst == NULL) {
        s = (BSTree)malloc(sizeof(BSTNode));
        s->key = key;
        s->lchild = NULL;
        s->rchild = NULL;
        *bst = s;
    } else if (key < (*bst)->key) {
        InsertBST(&(*bst)->lchild, key);
    } else if (key > (*bst)->key) {
        InsertBST(&(*bst)->rchild, key);
    }
}
```

如果关键字相等，教材代码不再插入。这样可以保证树中关键字不重复。如果实际应用允许重复关键字，就需要约定重复值放在左子树、右子树，或者在结点中增加计数字段。

创建二叉排序树就是从空树开始，按输入顺序逐个插入关键字：

```
void CreateBST(BSTree *bst) {
    KeyType key;

    *bst = NULL;
    scanf("%d", &key);

    while (key != ENDKEY) {
        InsertBST(bst, key);
        scanf("%d", &key);
    }
}
```

这里有一个重要结论：**同一组关键字，如果输入顺序不同，生成的二叉排序树形态可能不同**。例如按接近均匀的顺序插入，树可能比较平衡；如果按递增或递减顺序插入，树会退化成单链表。

注意：二叉排序树的插入只能在叶子节点处修改。假如要修改内部，树的结构很与有可能改变，时间复杂度激增，个人认为是二叉排序树的问题，创建的时候最好是已经排好序的状态

二叉排序树的删除

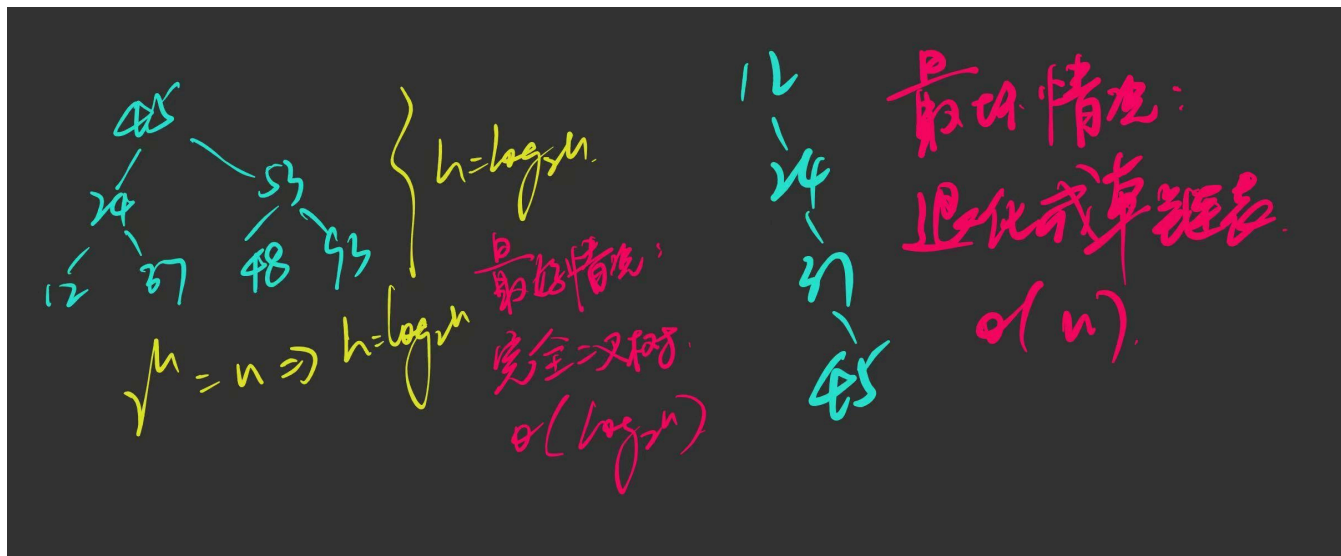
删除二叉排序树中的结点时，不能简单删掉以该结点为根的整棵子树，而要在删除后仍保持二叉排序树性质。设待删结点为 p ，通常分三种情况：

1. p 是叶子结点：直接删除。
2. p 只有一棵子树：让 p 的双亲直接指向这棵子树。
3. p 有两棵子树：用 p 的中序前驱或中序后继替代 p ，再删除那个前驱或后继结点。

用中序前驱处理时，核心思想是：在 p 的左子树中找到最右下的结点 s ，它是小于 p 的最大关键字。用 $s \rightarrow \text{key}$ 覆盖 $p \rightarrow \text{key}$ 后，再删除原来的 s 。因为 s 最多只有左子树，所以删除 s 会转化为前两种简单情况。

二叉排序树时间复杂度

二叉排序树的查找、插入和删除都与树高有关。若树高为 h ，这些操作的时间复杂度为 $O(h)$ 。最好或较平衡情况下， h 接近 $\log_2 n$ ；最坏情况下，树退化成单链表， $h = n$ ，时间复杂度退化为 $O(n)$ 。



2. 平衡二叉排序树

平衡二叉排序树又称 AVL 树。它首先是一棵二叉排序树，同时还要求任意结点的左子树和右子树高度差的绝对值不超过 1。

结点的平衡因子定义为：

$$bf = \text{左子树高度} - \text{右子树高度}$$

对于平衡二叉排序树，每个结点的平衡因子只能是 -1、0 或 1。引入 AVL 树的目的，是避免普通二叉排序树因插入顺序不佳而退化成单链表，从而提高查找效率。

AVL 树的查找过程与普通二叉排序树完全相同，区别主要出现在插入和删除后。如果一次插入导致某个结点的平衡因子变成 2 或 -2，就要通过旋转恢复平衡。

原则：

- 降低排序树高度
- 保持二叉排序树性质

教材中把失衡情况归纳为四类：

失衡类型	插入位置	调整方法	直观理解
LL 型	A 的左子树的左子树	对 A 做一次右旋	左边太高，向右转
RR 型	A 的右子树的右子树	对 A 做一次左旋	右边太高，向左转
LR 型	A 的左子树的右子树	先对 B 左旋，再对 A 右旋	先拉直，再调整
RL 型	A 的右子树的左子树	先对 B 右旋，再对 A 左旋	先拉直，再调整

其中 A 表示最低层失衡结点，B 是 A 较高一侧的孩子，C 是双旋时参与调整的孙子结点。

核心原则：寻找最低层失衡点，给三个数排序，第二大的点作为根节点，其余两点按照二叉排序树性质作为左右子树

LL 型右旋的核心代码可以理解为：

```
A->lchild = B->rchild;
B->rchild = A;
A->bf = 0;
B->bf = 0;
```

RR 型左旋与 LL 型对称：

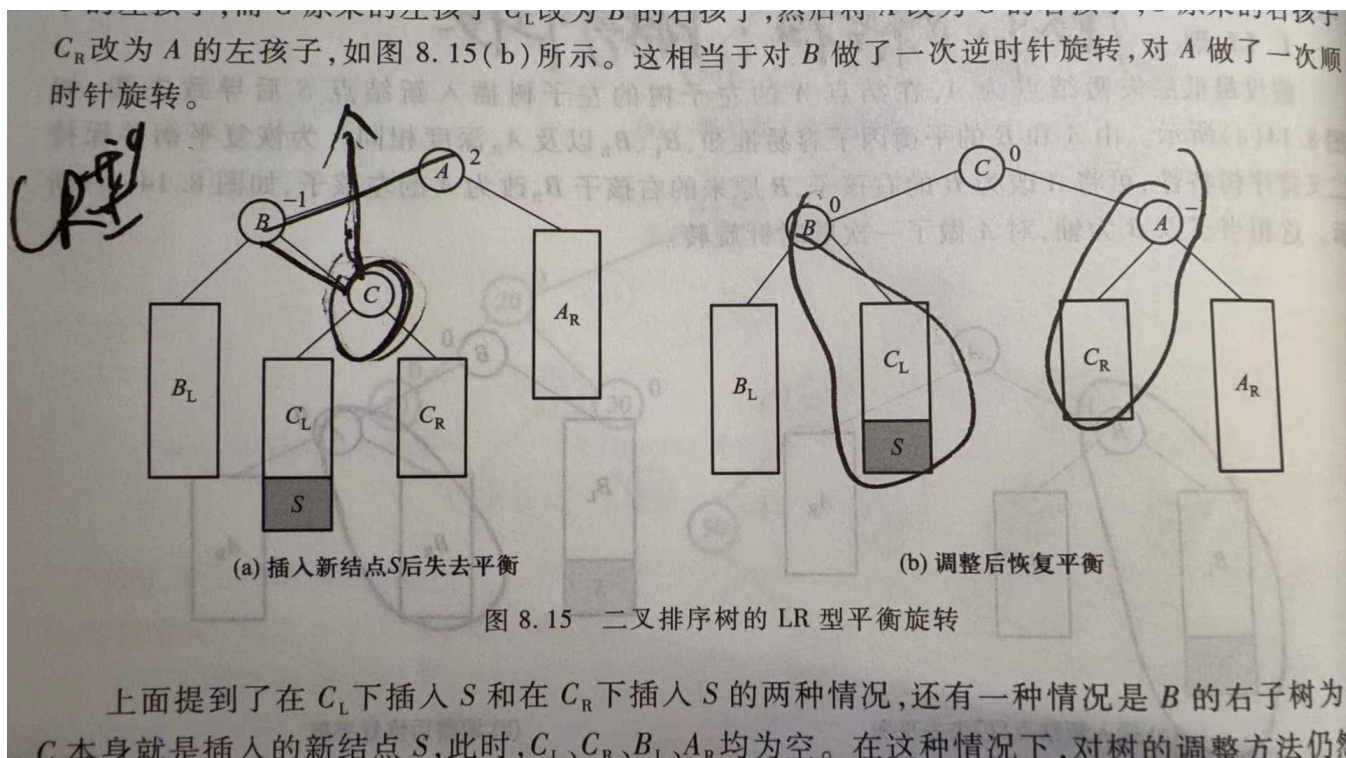
```
A->rchild = B->lchild;
B->lchild = A;
A->bf = 0;
B->bf = 0;
```

LR 型需要先把 B 和 C 调整，再把 A 和 C 调整：

```
B->rchild = C->lchild;
A->lchild = C->rchild;
C->lchild = B;
C->rchild = A;
```

RL 型与 LR 型对称：

```
B->lchild = C->rchild;
A->rchild = C->lchild;
C->lchild = A;
C->rchild = B;
```



旋转的目标不是改变二叉排序树的中序序列, 而是在保持中序有序的前提下降低树高。也就是说, 旋转前后关键字的大小关系不变, 只是树的形态变得更均衡。

AVL 树的插入过程可以概括为:

1. 按二叉排序树的方法查找插入位置。
2. 插入新结点。
3. 从插入位置向上更新平衡因子。
4. 找到最低层失衡结点。
5. 根据 LL、RR、LR、RL 四种类型进行旋转。

由于 AVL 树始终保持高度平衡, 它的树高为 $O(\log n)$, 因此查找、插入和删除的时间复杂度都是 $O(\log n)$ 。不过 AVL 树为了维护平衡, 需要额外保存平衡因子, 并在插入、删除时进行旋转, 所以实现复杂度高于普通二叉排序树。

二、哈希查找

哈希查找又称散列查找、杂凑查找或关键字地址计算法。它的基本思想是: 在关键字 k 和存储位置 p 之间建立一个函数关系

$$p = H(k)$$

其中 $H(k)$ 称为哈希函数。创建哈希表时, 把关键字为 k 的记录存入地址 $H(k)$; 查找时, 再通过同一个函数计算位置。如果没有冲突, 就可以接近常数时间完成查找。

哈希查找要解决两个核心问题: 如何构造哈希函数, 以及如何处理冲突。

1. 哈希函数的构造

构造哈希函数时，一般希望满足两个原则：第一，函数本身计算简单；第二，计算出的地址分布均匀，尽量减少冲突。

教材中常见的哈希函数构造方法有以下几种。

数字分析法

数字分析法适用于事先知道关键字集合，且关键字位数比哈希地址位数多的情况。通过分析关键字各位的分布情况，选取分布较均匀的若干位作为哈希地址。

例如一组 8 位十进制关键字中，如果第 3、4 位分布比较均匀，就可以取这两位作为地址。若选中的位分布很集中，就会产生大量冲突。

平方取中法

平方取中法先求关键字的平方，再取平方值中间的若干位作为哈希地址。因为平方后的中间几位通常与关键字的多位都有关，所以在无法判断哪些位分布均匀时，这种方法比较常用。

分段叠加法

分段叠加法按哈希地址所需位数，把关键字分成若干段，然后把各段相加，舍弃最高进位后的结果作为哈希地址。常见方式有移位叠加和折叠叠加。

除留余数法

除留余数法是最常见、最实用的方法。设哈希表长为 m ，选择一个不大于 m 的素数 p ，定义

$$H(k) = k \% p$$

例如表长为 13 时，可以取 $p = 13$ ，则关键字 46 的哈希地址为

$$H(46) = 46 \% 13 = 7$$

实际应用中， p 的选择会明显影响冲突情况。一般尽量选素数，并避免与关键字分布规律产生明显关系。

伪随机数法

伪随机数法采用伪随机函数作为哈希函数，即

$$H(key) = random(key)$$

这种方法适合关键字分布不明确的场景，但要保证同一关键字每次计算出的地址一致。

2. 冲突处理方法

不同关键字可能被映射到同一个哈希地址，这种现象称为冲突。发生冲突的关键字称为同义词。由于关键字集合通常远大于哈希表地址空间，冲突一般无法完全避免，只能尽量减少并妥善处理。

开放寻址法

开放定址法的基本思想是：如果初始地址 $H(key)$ 发生冲突，就按照某个增量序列继续寻找下一个地址，直到找到空单元。

通用形式为

$$H_i = (H(key) + d_i) \% m$$

其中 m 为表长， d_i 为第 i 次探测的增量。

线性探测再散列的增量为

$$d_i = 1, 2, 3, \dots, m - 1$$

它的特点是发生冲突后顺序查看下一个单元。优点是只要表未满，就一定能找到空位置；缺点是容易产生聚集现象，也就是一段连续单元被填满后，更多关键字会被挤到同一片区域。

二次探测再散列的增量通常为

$$d_i = 1^2, -1^2, 2^2, -2^2, \dots$$

它可以减少线性探测中的聚集，但不一定在所有情况下都能探测到表中每个位置。

伪随机探测再散列则使用伪随机数序列作为增量。它也可以减少聚集，但实现时要保证探测序列可重复。

再哈希法

再哈希法同时构造多个哈希函数：

$$H_1(key), H_2(key), \dots, H_k(key)$$

当一个哈希函数产生冲突时，就使用下一个哈希函数重新计算地址，直到不再冲突。这种方法不容易产生聚集，但会增加计算时间。

拉链法

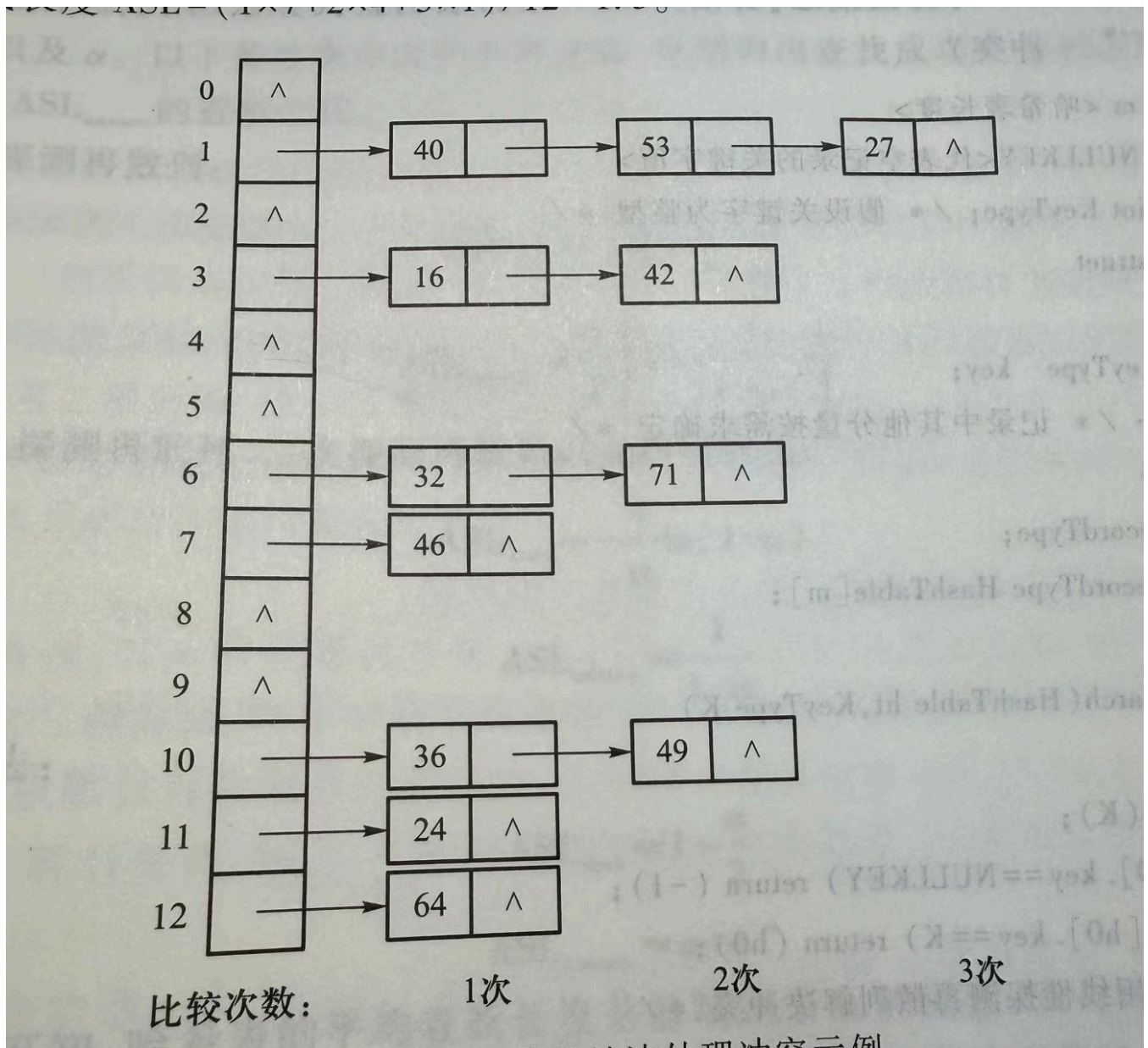
链地址法把所有哈希地址相同的记录放到同一个单链表中。哈希表的每个位置不直接存一个记录，而是存放对应同义词链的头指针。

例如，若 $H(32) = 6$ ， $H(45) = 6$ ， $H(58) = 6$ ，则这三个关键字都放在哈希表第 6 个位置对应的链表中。查找时先计算地址，再到该地址对应的链表里顺序查找。

链地址法适合经常插入和删除的情况。因为删除链表中的一个结点比较方便，不会像开放定址法那样影响后续探测路径。

优于开放寻址法

[哈希表 | xiaomao's blog](#)



建立公共溢出区

公共溢出区法把哈希表分成基本表和溢出表。没有冲突的记录放入基本表；凡是发生冲突的记录，统一放入溢出表。查找时先查基本表，再查溢出表。

这种方法思路简单，但如果冲突很多，溢出表会变长，查找效率会下降。

3. 哈希表的查找过程

哈希表的查找过程与建表过程一致。以开放定址法为例，查找关键字 k 的过程是：

1. 计算初始地址 $p = H(k)$ 。
2. 若 $ht[p]$ 为空，则查找失败。
3. 若 $ht[p].key == k$ ，则查找成功。
4. 若 $ht[p]$ 非空但关键字不等，则按冲突处理方法计算下一个地址。
5. 重复上述过程，直到成功、遇到空位置，或者探测完整张表。

用线性探测处理冲突时，查找代码可以写成：

```

#define NULLKEY -1
#define m 16

typedef struct {
    KeyType key;
    OtherType other_data;
} HashTable[m];

int Hash(KeyType key) {
    return key % 13;
}

int HashSearch(HashTable ht, KeyType key) {
    int p = Hash(key);
    int i = 0;

    while (ht[p].key != NULLKEY && ht[p].key != key && i < m) {
        p = (p + 1) % m;      /* 线性探测 */
        i++;
    }

    if (ht[p].key == key)
        return p;
    else
        return -1;
}

```

这里的 `NULLKEY` 表示空单元。若采用开放定址法，删除记录时不能简单把单元置空，否则可能中断后续同义词的查找路径。通常需要设置一个特殊的“已删除”标记。

4. 哈希法性能分析

由于冲突存在，哈希查找并不一定只比较一次。评价哈希表性能时仍然使用平均查找长度。

影响哈希查找性能的主要因素有三个：

1. 哈希函数是否均匀。
2. 冲突处理方法是否合适。
3. 哈希表的装填因子。

装填因子 α 定义为：

$$\alpha = \frac{\text{表中记录数}}{\text{哈希表长度}}$$

装填因子越大，哈希表越满，发生冲突的可能性越高；装填因子越小，冲突越少，但空间浪费越多。

在哈希函数均匀的前提下，哈希表的平均查找长度主要取决于装填因子和冲突处理方法。一个重要结论是：哈希表的平均查找长度主要是 α 的函数，而不是记录总数 n 的函数。因此，即使数据规模很大，只要合理控制装填因子，哈希查找仍可以保持较高效率。

对于具体哈希表，通常可以直接根据每个关键字的查找比较次数计算成功查找的 ASL：

$$ASL_{成功} = \frac{1}{n} \sum_{i=1}^n c_i$$

查找失败的 ASL 则要从每个可能的哈希入口出发，计算从该入口开始直到确定失败需要比较的次数，再求平均。

三、查找方法对比

查找方法	存储结构	是否要求有序	核心思想	平均时间复杂度	插入删除	适用场景
二叉排序树	二叉链表	树中保持有序关系	按大小走左/右子树	平均 $O(\log n)$, 最坏 $O(n)$	较方便	动态查找表
平衡二叉排序树	二叉链表	树中保持有序关系和平衡	查找同 BST, 插入删除后旋转	$O(\log n)$	需要维护平衡	动态查找且要求稳定效率
哈希查找	哈希表	不要求关键字有序	由哈希函数计算地址	平均接近 $O(1)$	视冲突方法而定	快速按关键字访问

从表中可以看出，二叉排序树适合动态查找，但性能受树形影响；平衡二叉排序树通过旋转维护平衡，避免退化；哈希查找平均效率最高，但它牺牲了关键字之间的有序关系，并且需要处理冲突。

四、综合分析

本阶段学习的查找算法可以围绕两条主线理解。

- 第一条主线是树形查找。二叉排序树把有序表中的大小关系转化成树上的左右分支，查找、插入、删除都沿一条路径进行，因此适合动态查找表。但普通二叉排序树的形态受插入顺序影响，可能退化成单链表。平衡二叉排序树通过平衡因子和旋转控制树高，使最坏情况下也能保持 $O(\log n)$ 的效率。
- 第二条主线是计算式查找。哈希查找不再通过多次比较逐步缩小范围，而是直接由关键字计算地址。它的理想情况是一次定位，但现实中必须处理冲突。哈希函数越均匀、装填因子越合理、冲突处理越合适，查找效率就越高。
- 实际选择查找方法时，需要结合表的规模、是否频繁插入删除、是否需要按序遍历等因素。如果需要动态维护有序数据，可以使用二叉排序树或平衡二叉排序树；如果只追求按关键字快速访问，而不关心顺序关系，哈希表通常更有优势。